G1 Robot — Navigation & Data-Logging Cheat-Sheet

Unitree G1 “Air” · controlled through the iPhone app's WebView over USB · we read its SLAM, drive it, and log every run to **dataset/**.

■ TODAY'S OBJECTIVE

Let the robot navigate with **Unitree's own (native) navigation**, while **we record the full dataset** (trajectory, telemetry, IMU, collisions, point clouds, camera):

```
python g1_goto.py benchmark B viz
```

Here the **firmware drives**; our script only watches and logs. (For comparison, **python g1_goto.py gotoviz B** runs *our own* A*/DWA algorithm instead.)

1 · Before every session (setup)

1. Charge the robot to **>80%** and stand it up. Keep the remote in hand (**L2 + B = stop**). Clear 2–3 m; keep people **behind** the robot (out of the laser).

2. iPhone: open the Unitree app, connect, **load the map** (Qw_20260625) and **relocalize**. Be on the **operation** screen, camera on, Web Inspector enabled.

3. Connect the iPhone to the laptop by **USB** and trust it.

4. Terminal 1 — start the bridge and leave it running: `ios_webkit_debug_proxy`

5. Terminal 2 — go to the project folder; run the commands below from there.

2 · Commands (run as `python g1_goto.py <cmd>`)

Command	What it does	Moves robot?
`reloccheck`	Read-only setup check: streams the robot's pose + which data is arriving. Nudge the robot and confirm the pose follows it. Always run this first.	No
`listwp`	List the saved waypoints (A, B, ...).	No
`waypoint A`	Capture a destination: drive the robot to the spot, then Ctrl+C → saves the last pose as 'A'. Repeat for B. Do this in the SAME relocalized session as the map.	You drive it
`buildmap 150 loc`	Build a clean reference map by accumulating the live laser while you drive slowly (through the open door into room B). Ctrl+C saves. Filters phantom/dynamic noise.	You drive it
`turntest`	Diagnostic: turns the robot in place to check the steering sign (fixes the 'spins forever' bug). Run before our own navigation.	Yes (in place)
`benchmark B viz`	★ OBJECTIVE: NATIVE Unitree navigation to B + live window, while we log the full dataset. Firmware drives; we record.	Yes (firmware)
`gotoviz B`	OUR navigation (A*/DWA) to B + live window, logging the dataset.	Yes (our code)
`tablecheck`	Capture a 3D cloud + camera photo in front of the robot (e.g. to confirm an invisible table). Diagnostic.	No
`posedump`	Dump raw pose / telemetry to discover what the robot exposes (battery, IMU, errorCode).	No

3 · What every run saves (in dataset/)

One JSON per trip (e.g. `..._native_B.json`) with: **samples** (~10 Hz: position, heading, speed, clearance, command, localization-match); **telemetry** (~1 Hz: battery, CPU, motor temps, IMU accel/gyro/rpy/leg-torque); **events** (collisions, relocalization jumps); **laser snapshots**; plus a 3D point cloud and a **camera photo** at every collision; and a **summary** (time, path length, efficiency, collisions, min clearance).

4 · Safety & stopping

• Remote always in hand: **L2 + B** stops the robot instantly. • To stop a script: close the window or press **Ctrl+C**. • If it spins, drifts, or heads at a wall → stop. • Don't run on a low battery (it degrades balance). • Keep the robot's head level so the LiDAR scans correctly.

Typical order: reloccheck → waypoint A → waypoint B → buildmap 150 loc → benchmark B viz (native + logging). Note: tables/glass are invisible to the head LiDAR — watch the robot.